

目录

- N 路组相连 Cache 相比直接映射 Cache 的改动
- Cache 工作流程与状态机转移逻辑
- Cache 结构设计
- 替换策略分析

N 路组相连 Cache 相比直接映射 Cache 的改动

- Tag/Data BRAM 由固定例化 1 对改为通过 generate 块例化 N 对；基于 Way 数量的模块内数据线和寄存器都需要相应扩为 N 条
- Cache Hit 的判断逻辑由单 Way 的 Hit 信号直接连出改为综合所有 Way 的 Hit 信号进行按位或得到全局的 Hit 信号
- 直接映射 Cache 由于每个 Cache Set 中只有一个 Way，因此发生 Cache Miss 时不需要使用替换策略；而 N 路组相连 Cache 需要采用一个特定的替换策略选择在发生 Cache Miss 时需要替换哪一个 Way，并且 Cache 的性能会**受到选用的替换策略的影响**
- 值得注意的是，即使在直接映射 Cache 中，由于 **Cache Set 的定位只依靠 Index 段**，仍然需要通过 Tag BRAM 来判定当前 Way 上存储的数据是否是目标访存地址上的数据（**比较 Tag BRAM 数据与访存地址的 Tag 段**）

Cache 工作流程与状态机转移逻辑

Cache 模块采用写回写分配（Write-back & Write-allocate）策略，使用五状态的状态机控制 Cache 的工作流，配合缓冲寄存器和控制信号完成访存请求和必要的主存数据同步。状态机及详细工作细节如下：

一、各状态下 Cache 主要完成的工作

- IDLE
 - **主要工作**：等待 CPU 新的读写请求；在上一条访存指令 Cache Miss 时，完成 Refill 处理并将访存结果前递给 CPU
 - **使用的信号与寄存器**：
 - addr_buf_we 置 1，锁存新的 CPU 访存请求
 - miss 置 0，解除 CPU 停顿状态
 - refill 指示换入操作。在 refill=1 时：
 - 使用 data_we[replace_way] 和 tag_we[replace_way] 覆写选定的 Way

- 将该 Way 标识为有效 `w_valid = 1`
- 对于写换入过程，由于已经提前在写入线 `w_line` 上完成覆写，因此直接置脏指示 `w_dirty = 1`
- 在数据写入 Cache 的周期，直接从传入 Cache 的写入数据将访存结果前递给 CPU
`data_from_mem = 1`

```
miss = 1'b0;           // 解除 CPU 停顿状态
addr_buf_we = 1'b1;    // 继续缓存下一个访存请求

if (refill) begin      // 在 IDLE 状态完成换入操作
    data_from_mem = 1'b1; // 把将要换入 Cache 的数据前递给 CPU
    w_valid = 1'b1;      // 标识 Refill 后的 Cache Line 可用
    w_dirty = 1'b0;      // 标识 Refill 后的 Cache Line 非 dirty
    data_we[replace_way] = 1'b1; // 选中需要覆写的 Way
    tag_we[replace_way] = 1'b1;
    if (op_buf) begin
        w_dirty = 1'b1; // 写换入，在换入过程中已经在 ret_buf 基础上执行了写入操作
    end
end
```

• READ

- **主要工作：**检查 CPU 读请求是否 Cache Hit，据此直接读出数据或执行 Cache Miss 处理
- **使用的信号与寄存器：**
 - `hit, miss`：若 Cache Hit，则直接获取结果并将 `miss` 置 0，`addr_buf_we=1` 继续接纳下一个请求流水；若 Cache Miss，则将 `miss` 置 1 使 CPU 停顿，同时锁存当前访存指令
`addr_buf_we=0`
 - `curr_dirty` 指示将被替换的 Cache Line 是否处于 dirty 状态；如果处于 dirty 状态，则需要先换出再换入

```

data_from_mem = 1'b0;           // 从 Cache 中把数据读出给 CPU
if (hit) begin                  // Cache Hit
    miss = 1'b0;                // CPU 继续流水执行
    addr_buf_we = 1'b1;         // 继续缓存下一个访存请求
end
else begin                      // Cache Miss
    miss = 1'b1;                // 停顿 CPU
    addr_buf_we = 1'b0;         // 锁存当前访存请求
    if (curr_dirty) begin      // 如果将被替换的 Cache Line 处于 dirty 状态，需要先换出
        // 准备好写入内存的地址和数据
        mem_w = 1'b1;
        mem_addr = dirty_mem_addr;
        mem_w_data = r_line[replace_way];
    end
end
end

```

- **WRITE**

- **主要工作：**检查 CPU 写请求是否 Cache Hit，据此直接写入数据或执行 Cache Miss 处理
- **使用的信号与寄存器：**
 - Cache Hit 时，同 READ 保持 miss=0 与 addr_buf_we=1，对命中的 Way 打开 BRAM 写使能 data_we[hit_way_id] = 1, tag_we[hit_way_id] = 1，并将该 Way 标记可用、脏状态 (w_valid = 1, w_dirty = 1)
 - Cache Miss 时，miss 置 1 停顿 CPU，通过 curr_dirty 判定脏状态并决定是否执行换出

```

data_from_mem = 1'b0;
if (hit) begin          // Cache Hit
    miss = 1'b0;                // CPU 继续流水执行
    addr_buf_we = 1'b1;        // 继续缓存下一个访存请求
    // 由于 Tag BRAM 覆写过程会同时重写 Way 上的 valid 和 dirty 位
    // 因此需要预先对 w_valid 和 w_dirty 寄存器赋正确的值
    w_valid = 1'b1;
    w_dirty = 1'b1;
    data_we[hit_way_id] = 1'b1; // 选中将要写入的 Way
    tag_we[hit_way_id] = 1'b1;
end
else begin              // Cache Miss
    miss = 1'b1;        // 停顿 CPU
    addr_buf_we = 1'b0; // 锁存当前访存请求
    if (curr_dirty) begin // 如果将被替换的 Cache Line 处于 dirty 状态，需要先换出
        // 准备好写入内存的地址和数据
        mem_w = 1'b1;
        mem_addr = dirty_mem_addr;
        mem_w_data = r_line[replace_way];
    end
end
end

```

- **MISS**

- **主要工作：**访问内存读取 Cache Line 执行换入操作
- **使用的信号与寄存器：**
 - mem_r 置 1 指示读内存操作
 - mem_addr 连线 addr_buf 向内存提供访存地址
 - miss=1 继续维持 CPU 停顿状态
 - mem_ready 指示内存是否就绪（在本状态下具体指内存读操作是否完成）；若内存已就绪，则将 ret_buf_we 置 1，把内存读出数据存入缓冲寄存器 ret_buf，同时 mem_r 置 0 撤销内存读请求状态
 - refill 置 1，指示回到 IDLE 状态时需要完成 Refill 操作

```

miss = 1'b1;                                // 维持 CPU 停顿状态
// 向内存发出读请求，从基址开始读入一个完整 Cache Line
mem_r = 1'b1;
mem_addr = addr_buf;
if (mem_ready) begin
    // 内存读出完成，关闭读请求并锁存读出数据
    mem_r = 1'b0;
    ret_buf_we = 1'b1;
end

```

- **W_DIRTY**

- **主要工作：**执行换出操作，将需要被替换且处于 dirty 状态的 Cache Line 写回内存
- **使用的信号与寄存器：**
 - miss=1 继续维持 CPU 停顿状态
 - mem_w 置 1 指示写内存操作
 - mem_addr 连线 dirty_mem_addr_buf 向内存提供访存地址
 - mem_w_data 连线 dirty_mem_data_buf 向内存提供写入数据
 - mem_ready 指示内存是否就绪（在本状态下具体指内存写操作是否完成）；若内存已就绪，则 mem_w 置 0 撤销内存写请求状态

```

miss = 1'b1;                                // 维持 CPU 停顿状态
mem_w = 1'b1;                                // 向内存发出写请求，执行换出
// 准备好写入内存的地址和数据
mem_addr = dirty_mem_addr_buf;
mem_w_data = dirty_mem_data_buf;
if (mem_ready) begin
    // 内存写入完毕，关闭写请求
    mem_w = 1'b0;
end

```

二、各状态转换条件与主要逻辑判定

1. IDLE 状态转移方向：

- 读访存指示 r_req 为 1 时跳转到 READ 状态
- 写访存指示 w_req 为 1 时跳转到 WRITE 状态
- 无新的访存命令时，维持 IDLE 状态

2. READ 及 WRITE 状态转移方向：

- 若 Cache Hit，则不需要执行 Refill 操作，可直接跳转到 READ/WRITE 状态执行下一条访存指令
- 若 Cache Miss 且将要被替换的行处于 dirty 状态，则跳转到 W_DIRTY 状态执行换出操作
- 若 Cache Miss 且将要被替换的行非 dirty，则跳转到 MISS 状态执行换入操作

3. MISS 状态转移方向：

- 若内存就绪（数据稳定读出），则跳转到 IDLE 状态完成 Refill
- 若内存仍未就绪，则保持 MISS 状态

4. W_DIRTY 状态转移方向：

- 若内存就绪（数据成功写入），则跳转到 MISS 状态执行换入操作
- 若内存仍未就绪，则保持 W_DIRTY 状态

```

case(current_state)
  IDLE: begin
    if (r_req) begin          // 读访存指令
      next_state = READ;
    end
    else if (w_req) begin      // 写访存指令
      next_state = WRITE;
    end
    else begin
      next_state = IDLE;
    end
  end
  READ: begin
    // Cache Miss, 且将被覆盖的 Cache Line 不需要换出
    // curr_dirty 信号的产生是独立的, 始终有效指示将在必要时被替换掉的 Cache Line 状态
    if (miss && !curr_dirty) begin
      next_state = MISS;
    end
    else if (miss && curr_dirty) begin // Cache Miss 且将被覆盖的 Cache Line 需要换出
      next_state = W_DIRTY;
    end
    else if (r_req) begin          // 新一条读访存指令
      next_state = READ;
    end
    else if (w_req) begin          // 新一条写访存指令
      next_state = WRITE;
    end
    else begin                    // 没有新的访存指令, 返回 IDLE 状态执行被挂起的换入操作
      next_state = IDLE;
    end
  end
  MISS: begin
    if (mem_ready) begin // 内存读出完成, 返回 IDLE 状态完成换入过程
      next_state = IDLE;
    end
    else begin // 等待内存读出
      next_state = MISS;
    end
  end
  WRITE: begin
    if (miss && !curr_dirty) begin // Cache Miss 且将被覆盖的 Cache Line 不需要换出
      next_state = MISS;
    end
  end
end

```

```

    else if (miss && curr_dirty) begin // Cache Miss 且将被覆盖的 Cache Line 需要换出
        next_state = W_DIRTY;
    end
    else if (r_req) begin // 新一条读访存指令
        next_state = READ;
    end
    else if (w_req) begin // 新一条写访存指令
        next_state = WRITE;
    end
    else begin // 没有新的访存指令，返回 IDLE 状态执行被挂起的换入操作
        next_state = IDLE;
    end
end
W_DIRTY: begin
    if (mem_ready) begin // 换出完成，跳转到 MISS 状态开始换入
        next_state = MISS;
    end
    else begin // 等待内存写入完成
        next_state = W_DIRTY;
    end
end
default: begin // 非法状态，跳转到 IDLE
    next_state = IDLE;
end
endcase

```

Cache 结构设计

一、Cache 主要结构和 Cache Hit 过程细节

Cache 主要结构：

- Cache 采用 Way 级别的 BRAM 对例化实现，在 generate 块中，每一个 Way 对应例化一个 Tag BRAM 和一个 Data BRAM
- 每个 BRAM 的地址线宽度为 INDEX_WIDTH，其中地址为 i 的 BRAM 对应编号为 i 的 Cache Set
- 每个 Way 上的 BRAM 都有自己独立的查询输入、结果输出线、Hit 状态位，用于在 Cache 中**并行查找**一个 Set 中的各个 Way

Cache 访问过程

1. 在一次访存过程中，在访存指令传入的周期，通过 `r_index` 立刻截取访存指令的 Index 段并传入 BRAM 并行读取该 Set 中的每个 Way 上的 Tag 和 Data 数据；同时，利用 `addr_buf` 在下个时钟周期缓存当前的访存指令
2. 在下一个时钟周期，通过 `addr_buf` 截取访存指令的 Tag 段，与从 Tag BRAM 读出的各个 Way 上的 Tag 段进行比对，产生每个位上的 Hit 信号 `hit_way[i]`
3. 通过一个 16 位宽**定长**线组 `hit_vec` 转存 `hit_way` 上的信息，再由此提取出全局 Cache Hit 信号 `hit`；同时，通过判断每一个 Way 上的 Hit 信号，找到真正 Hit 的 Way 编号 `hit_way_id`

```

assign r_index = addr[INDEX_WIDTH+LINE_OFFSET_WIDTH+SPACE_OFFSET - 1: LINE_OFFSET_WIDTH+SPACE_OI
assign w_index = addr_buf[INDEX_WIDTH+LINE_OFFSET_WIDTH+SPACE_OFFSET - 1: LINE_OFFSET_WIDTH+SPA
assign tag = addr_buf[31:INDEX_WIDTH+LINE_OFFSET_WIDTH+SPACE_OFFSET];
assign word_offset = addr_buf[LINE_OFFSET_WIDTH+SPACE_OFFSET-1:SPACE_OFFSET];

genvar i;
generate
    for (i = 0; i < WAY_NUM; i = i + 1) begin : ways
        bram #(
            .ADDR_WIDTH(INDEX_WIDTH),
            .DATA_WIDTH(TAG_WIDTH + 2)          // 从高位到低位依次为 valid 位、dirty 位、Tag 数据
        ) tag_bram(
            .clk(clk),
            .raddr(r_index),
            .waddr(w_index),
            .din({w_valid, w_dirty, tag}),
            .we(tag_we[i]),
            .dout({valid[i], dirty[i], r_tag[i]}) // 每个 Way 对应的查询结果
        );

        bram #(
            .ADDR_WIDTH(INDEX_WIDTH),
            .DATA_WIDTH(LINE_WIDTH)
        ) data_bram(
            .clk(clk),
            .raddr(r_index),
            .waddr(w_index),
            .din(w_line),          // 写入 Cache 的数据
            .we(data_we[i]),      // 只有特定的 Cache Set 才能被写入
            .dout(r_line[i])
        );

        // Cache Hit 逻辑: Cache Line 有效, 且 Tag 校验通过
        assign hit_way[i] = valid[i] && (r_tag[i] == tag);
    end
endgenerate

wire [15 : 0] hit_vec;          // Cache Hit 状态向量, hit_vec[i] 指示第 i 个 Way 是否 Cache Hit
generate
    for (i = 0; i < WAY_NUM; i = i + 1) begin : hit_assign
        assign hit_vec[i] = hit_way[i];
    end
    for (i = WAY_NUM; i < 16; i = i + 1) begin : zero_assign

```

```

        assign hit_vec[i] = 1'b0;
    end
endgenerate

// 总 Cache Hit 信号
assign hit = |(hit_vec & ((1<<WAY_NUM)-1));    // &(1<<WAY_NUM)-1 用于排除 hit_vec 中的无效高位,

integer j;
always @(*) begin
    hit_way_id = 0;
    for (j = 0; j < WAY_NUM; j = j + 1) begin
        // 找到真正发生 Cache Hit 的 Way, 将其编号赋值给 hit_way_id
        // 只截取循环变量 j 的低位 (有效位)
        if (hit_way[j]) hit_way_id = j[WAY_NUM_WIDTH-1:0];
    end
end
end

```

二、Cache 换出细节

- 使用 curr_dirty 指示将要被替换的 Cache Line 的 dirty 状态, 如果 curr_dirty 为 1, 则需要执行换出操作
- 通过 Tag BRAM 中读出的 curr_r_tag 和传入访存指令中指示的 w_index 拼接得出换出过程的内存写入基址 dirty_mem_addr
- 在 READ/WRITE 状态下, 如果判定 Cache Miss 且需要换出的 Cache Line 处于 dirty 状态, 则在下一个时钟周期将内存写入基址和数据缓存到 dirty_mem_addr/data_buf 中用于换出

```

wire curr_dirty = dirty[replace_way]; // 将被替换的 Way 的 dirty 指示
wire [TAG_WIDTH-1:0] curr_r_tag = r_tag[replace_way]; // 将被替换的 Way 的 Tag 段

wire [31:0] dirty_mem_addr = {curr_r_tag, w_index} << (LINE_OFFSET_WIDTH+SPACE_OFFSET);

reg [31:0] dirty_mem_addr_buf; // 换出地址缓冲寄存器
reg [127:0] dirty_mem_data_buf; // 换出数据缓冲寄存器
always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        dirty_mem_addr_buf <= 0;
        dirty_mem_data_buf <= 0;
    end
    else begin
        if ((current_state == READ || current_state == WRITE) && !hit && curr_dirty) begin
            // Cache Miss 且将被覆盖的 Cache Line 处于 dirty 状态
            dirty_mem_addr_buf <= dirty_mem_addr;
            dirty_mem_data_buf <= r_line[replace_way];
        end
    end
end
end

```

三、Cache 写入细节

- Cache 写入分为三种情形：写换入、读换入、写访存
- 在写换入情形下，需要写入的内存不在 Cache 中，需要先**对内存读出数据进行覆盖写入**，再写入 Cache
- 在读换入情形下，需要读取的内存不在 Cache 中，只需要**将内存读出数据直接写入** Cache 即可
- 在写访存情形下，需要写入的内存已在 Cache 中，**将 Cache 读出数据中部分内容覆盖后再写入** Cache 原位即可

```

wire [LINE_WIDTH-1:0] curr_r_line = r_line[hit_way_id];
assign w_line_mask = 32'hFFFFFFFF << (word_offset*32); // 写掩码，只修改目标内存地址上的数据
assign w_data_line = w_data_buf << (word_offset*32); // 将新数据移位对齐

assign w_line = (current_state == IDLE && op_buf) ? ret_buf & ~w_line_mask | w_data_line : // 写换入
                (current_state == IDLE) ? ret_buf : // 读换入
                curr_r_line & ~w_line_mask | w_data_line; // 写访存

```

四、Cache 与内存交互细节

- 在流水线 CPU 正常工作时，`addr_buf_we` 信号为 1，将当前访存指令的访存信息在下一个时钟周期缓存到 `addr_buf`, `w_data_buf`, `op_buf` 中以便判定 Cache Hit
- 当内存读就绪时，`ret_buf_we` 信号有效，内存读出的 Cache Line 在下一个时钟周期缓存到 `ret_buf` 以便换入和进行写访存操作
- 在 MISS 状态且内存读就绪后，将在下一个时钟周期跳转到 IDLE 状态执行 Refill 操作，因此将 `refill` 信号在下一个时钟周期置 1；由于 Refill 固定需要一个时钟周期完成，因此在 IDLE 状态将 `refill` 信号在下一个时钟周期置 0

```
always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        addr_buf <= 0;
        ret_buf <= 0;
        w_data_buf <= 0;
        op_buf <= 0;
        refill <= 0;
    end
    else begin
        if (addr_buf_we) begin // 暂存当前指令的访存请求信息
            addr_buf <= addr;
            w_data_buf <= w_data;
            op_buf <= w_req;
        end
        if (ret_buf_we) begin // 暂存主存读出的 Cache Line
            ret_buf <= mem_r_data;
        end

        if (current_state == MISS && mem_ready) begin // 主存读出数据稳定后，拉起 refill 信号
            refill <= 1;
        end
        if (current_state == IDLE) begin // IDLE 状态执行完 Refill 后，消除 refill 信号
            refill <= 0;
        end
    end
end
```

五、Cache 对 CPU 的数据提供细节

- 结合 `addr_buf` 缓存的访存地址中截取出的 `word_offset` 段，分别从 Cache 读出数据 `curr_r_line` 和内存读出数据 `ret_buf` 中**截取出单字的数据**
- 根据 `data_from_mem` 控制信号，选择选用内存数据还是 Cache 数据提供给 CPU

- 当内存数据未准备好时，`data_from_mem` 仍为 0，但由于 Cache Miss 状态成立，`r_data` 会直接接 0，避免在与 CPU 的数据交互线上出现无效数据

```
always @(*) begin
    if (word_offset < (1 << LINE_OFFSET_WIDTH)) begin // 检查 word_offset 是否合法
        // 从 (word_offset + 1) * DATA_WIDTH - 1 位开始向低位截取 DATA_WIDTH 位
        cache_data = curr_r_line[(word_offset + 1) * DATA_WIDTH - 1 -: DATA_WIDTH];
        mem_data = ret_buf[(word_offset + 1) * DATA_WIDTH - 1 -: DATA_WIDTH];
    end
    else begin
        cache_data = 0;
        mem_data = 0;
    end
end

// 选择提供给 CPU 的数据；如果 Cache Miss 且内存数据未准备好，则直接接 0
assign r_data = data_from_mem ? mem_data : hit ? cache_data : 0;
```

替换策略分析

各种替换策略的具体实现方式

1. LRU 替换策略

核心思想：在每个 Cache Set 中建立 Way 与年龄之间的双射，一个 Way 的年龄越大，代表它未被访问的时间越久；选择年龄最大的 Way 用于替换。

使用的辅助变量：用于记录每个 Set 中每个 Way 的年龄

```
reg [WAY_NUM_WIDTH-1:0] lru_age [0:SET_NUM-1][0:WAY_NUM-1];
```

- 初始化时，即使所有 Way 都没有使用，也为编号为 *i* 的 Way 赋予年龄 *i*，从而**保证在 Cache 运行的任何时刻所有 Way 的年龄都彼此不同**

```
for (s = 0; s < SET_NUM; s = s + 1) begin
    for (w = 0; w < WAY_NUM; w = w + 1) begin
        lru_age[s][w] <= w;
    end
end
```

- 当发生 Cache Hit 时，将对应的 Way 的年龄清零，其余所有 Way 的年龄都加 1
- 当发生 Cache Miss 时，选择年龄最大的 Way 用于本次替换

```
for (j = 0; j < WAY_NUM; j = j + 1) begin
    if (lru_age[w_index][j] == WAY_NUM - 1) begin
        replace_way = j[WAY_NUM_WIDTH-1:0];
    end
end
```

- 换入（Refill）完成后，将该 Way 的年龄清零，其余所有 Way 年龄加 1

```
for (w = 0; w < WAY_NUM; w = w + 1) begin
    lru_age[w_index][w] <= lru_age[w_index][w] + 1;
end
lru_age[w_index][replace_way] <= 0;
```

2. FIFO 替换策略

核心思想：选择 Cache Set 中最早存入数据的 Way 用于替换

使用的辅助变量：用于记录每个 Set 中最早存入数据的 Way 编号

```
reg [WAY_NUM_WIDTH-1:0] fifo_ptr [0:SET_NUM-1];
```

- 初始化时，将 0 号 Way 作为替换目标

```
for (s = 0; s < SET_NUM; s = s + 1) begin
    fifo_ptr[s] <= 0;
end
```

- 发生 Cache Miss 时，选择队首的 Way 用于本次替换

```
replace_way = fifo_ptr[w_index];
```

- 换入完成后，将替换目标指向编号上的后继 Way；可以保证 Way 按编号顺序存入和使用

```
fifo_ptr[w_index] <= fifo_ptr[w_index] + 1;
```

3. 伪随机替换策略

核心思想：通过伪随机算法选择一个 Way 用于替换

使用的辅助变量：指向下一个将要换出的 Way 的编号；不论哪个 Set 需要换出，都对此变量进行使用和更新

```
reg [WAY_NUM_WIDTH-1:0] rand_ptr;
```

- 初始化时，将 0 号 Way 作为替换目标

```
rand_ptr <= 0;
```

- 发生 Cache Miss 时，选择编号为 rand_ptr 的 Way 用于本次替换

```
replace_way = rand_ptr;
```

- 换入完成后，执行 rand_ptr <= rand_ptr + 1。由于不同 Cache Set 的 Miss 频率与时机无法预测，rand_ptr 在各个 Cache Set 的选择客观上形成了伪随机性

```
rand_ptr <= rand_ptr + 1;
```

4. LFU 替换策略

核心思想：选择累计访问次数最少的 Way 用于替换

使用的辅助变量：记录每个 Set 中每个 Way 的累计访问次数

```
reg [7:0] lfu_cnt [0:SET_NUM-1][0:WAY_NUM-1];
```

为每个 Set 中的每个 Way 使用一个 8 位的访问计数器，用于记录各个数据块被访问的频率。

- 初始化时，将每个 Way 的累计访问次数归零

```
for (s = 0; s < SET_NUM; s = s + 1) begin
    for (w = 0; w < WAY_NUM; w = w + 1) begin
        lfu_cnt[s][w] <= 0;
    end
end
```

- 发生 Cache Hit 时，将对应的 Way 的累计访问次数加 1；如果累计访问次数达到统计上界，就不再更新

```
if (lfu_cnt[w_index][hit_way_id] < 8'hFF) begin
    lfu_cnt[w_index][hit_way_id] <= lfu_cnt[w_index][hit_way_id] + 1;
end
```

- 发生 Cache Miss 时，选择累计访问次数最少的 Way 用于本次替换

```
reg [7:0] min_lfu;
integer w;
min_lfu = 8'hFF;
replace_way = 0;
for (w = 0; w < WAY_NUM; w = w + 1) begin
    if (lfu_cnt[w_index][w] <= min_lfu) begin
        min_lfu = lfu_cnt[w_index][w];
        replace_way = w[WAY_NUM_WIDTH-1:0];
    end
end
```

- 换入完成后，将该 Way 的累计访问次数重置为 1

```
lfu_cnt[w_index][replace_way] <= 1;
```

各种策略的测试表现

采用 MODE=1 进行测试，仿真参数：

- 访存次数：3000 读+3000 写
- Cache 配置：4 路组相连
- 跳转概率 0.1

仅改变替换策略，使用同一个 testbench 文件运行仿真，结果如下：

替换策略	Total Cycles	Miss Count
LRU	28756	1733
FIFO	28756	1733
Random	28896	1746
LFU	29578	1814

发现与期望的测试结果存在差异：LRU 策略与 FIFO 策略的表现极为接近（甚至可能出现 FIFO 策略优于 LRU 策略的情况），LFU 策略与其它三种策略的性能差异不大。

- 在 MODE=1 时，访问模式可以归结为“长序列顺序访问+小概率跳转”，具有很强的空间局部性，但**欠缺时间局部性**。被读入的 Cache Line 短期内被访问之后就完全不再被访问，导致 LRU 策略“优先替换使用次数最少的 Cache Line”实际上近似退化为 FIFO 策略

- 此外，由于访问的顺序性较强，**所有内存区域的访问次数大多相同**，因此 LFU 策略对历史访问次数最多的 Cache Line 的保护偏好不能充分体现，从而没有充分体现 LFU 策略相较于其它三种策略在实际工作场景下的性能劣势
- 为解决这些问题，引入 `MODE=2`，以更好地模拟真实 CPU 工作场景下的 Cache 访问模式。具体的工作特性包括：
 - 程序在一定时间范围内，对于某一块连续内存区域（**工作集**）的访问频率较高（循环、函数调用等）
 - 程序的访问热点可能在一段时间后从一块连续内存转向另一块连续内存区域
 - 程序执行过程中可能随时访问少量的独立内存位置（访问全局变量、系统中断等）
- `MODE=2` 采用三个参数模拟真实 CPU 工作场景：
 - `WORKING_SET_SIZE` 定义工作集的大小
 - `HOT_PROB` 定义访存区域落在工作集内的概率
 - `PHASE_LINES` 定义发生访问热点跳转的访存次数阈值

采用 `MODE=2` 重新进行测试，同样仅改变替换策略，使用同一个 `testbench` 文件运行仿真，结果如下：

替换策略	Total Cycles	Miss Count
LRU	17118	981
FIFO	21062	1267
Random	21478	1310
LFU	38046	2760

可见，在更贴近真实 CPU 运行场景的环境下，四种策略的性能体现出典型的性能差异： $LRU > FIFO \geq Random \gg LFU$ 。